Exercício: checkout com colaboradores simulados

Implemente e teste um serviço de finalização de compra de uma loja virtual. O serviço deve coordenar reserva de estoque, cobrança, registro do pedido e envio de confirmação. Esses quatro colaboradores representam sistemas externos e serão substituídos por **mocks do Mockito**. O serviço e os objetos de domínio devem ser reais.

A atividade foi planejada para duas aulas de aproximadamente 100 minutos, em duplas, com alternância entre quem escreve o teste e quem implementa. Pressupõe familiaridade com classes, interfaces, exceções, `BigDecimal` e testes simples no JUnit. O objetivo principal não é construir uma loja completa: é aprender a testar decisões e efeitos observáveis sem acessar banco, rede ou serviços reais.

1. O que você vai praticar

Ao final, você deverá saber configurar respostas e exceções de um mock, verificar chamadas e argumentos, verificar a ausência de interações, capturar um objeto enviado a um colaborador e verificar uma ordem relevante para o negócio. Também deverá distinguir uma asserção sobre o resultado de uma verificação sobre a colaboração entre objetos.

Neste exercício, um mesmo objeto criado pelo Mockito pode funcionar como **stub**, quando fornece uma resposta programada, e como **mock**, quando suas interações são verificadas. Não basta configurar `when(...).thenReturn(...)`: o teste precisa estabelecer o que o serviço deve produzir e quais efeitos são permitidos. As APIs de configuração, verificação e captura estão na documentação oficial do Mockito. `1`

2. Ambiente e projeto inicial

Use Java 21 ou superior, Maven, JUnit Jupiter e Mockito. O projeto inicial fixa Java `release 21`, JUnit `5.13.4` e Mockito `5.18.0` para tornar a atividade reproduzível; não se trata de uma exigência de utilizar as versões mais recentes.

Descompacte o projeto dos alunos e execute, na pasta que contém o `pom.xml`:

Bash

```
mvn test
```

Há um teste inicial de caracterização do requisito de entrada nula. Ele deve ficar **vermelho** porque `finalizar` ainda lança `UnsupportedOperationException`. A primeira tarefa é implementar

apenas o necessário para fazê-lo passar. As interfaces e os records já estão disponíveis; não precisam ser reescritos.

O `pom.xml` configura o Mockito como agente da JVM de testes, seguindo a orientação para o mock maker inline no Java 21+. Se executar diretamente pela IDE, prefira delegar a execução ao Maven para usar a mesma configuração. [1] O uso de `junit-jupiter` e do Maven Surefire segue o guia do JUnit. [2]

3. Contratos fornecidos

O pacote do projeto é `br.edu.checkout`. A entrada representa a compra de **um único SKU**, em uma quantidade informada. Não há carrinho com múltiplos itens, frete, desconto ou impostos.

Java

```
public record SolicitacaoCheckout(
    String clienteId,
    String email,
    String sku,
    int quantidade,
    BigDecimal precoUnitario
) {}

public record Pagamento(boolean aprovado, String transacaoId) {}

public record PedidoConfirmado(
    String clienteId,
    String email,
    String sku,
    int quantidade,
    BigDecimal total,
    String transacaoId
) {}

public record ResultadoCheckout(
    String numeroPedido,
    BigDecimal total,
    boolean notificacaoEnviada
) {}
```

Os colaboradores têm os seguintes contratos. Cada interface já está em seu próprio arquivo no projeto inicial.

Java

```java
public interface EstoqueGateway {
    boolean reservar(String sku, int quantidade);
    void liberar(String sku, int quantidade);
}

public interface PagamentoGateway {
    Pagamento cobrar(String clienteId, BigDecimal total);
}

public interface PedidoRepository {
    String salvar(PedidoConfirmado pedido);
}

public interface Notificador {
    void enviarConfirmacao(
        String email, String numeroPedido, BigDecimal total);
}
```

A classe a implementar é `ServicoCheckout`. Ela recebe esses quatro colaboradores por construtor e expõe:

Java

```java
public ResultadoCheckout finalizar(SolicitacaoCheckout solicitacao)
```

São fornecidas quatro exceções que estendem `RuntimeException`: `EstoqueInsuficienteException`, `PagamentoRecusadoException`, `FalhaPagamentoException` e `FalhaNotificacaoException`. As duas últimas recebem uma mensagem no construtor.

4. Regras que os testes devem especificar

R1 — Rejeitar entradas inválidas antes de qualquer efeito

Lance `IllegalArgumentException` se a solicitação for nula; se `clienteId`, `email` ou `sku` for nulo, vazio ou contiver somente espaços; se a quantidade for menor ou igual a zero; ou se o preço unitário for nulo, zero ou negativo.

Nesses casos, **nenhum dos quatro colaboradores pode ser chamado**. A validação pertence ao serviço, não aos records. Não é necessário validar o formato do e-mail nem alterar os textos recebidos. A mensagem da exceção não faz parte do contrato avaliado.

R2 — Calcular o valor exato

Calcule `precoUnitario × quantidade` com `BigDecimal`, sem passar por `double` e sem arredondar. Por exemplo, três unidades de `19.90` resultam em `59.70`; três unidades de `0.005` resultam em `0.015`.

O segundo exemplo é proposital: neste domínio didático, o preço pode ter mais de duas casas decimais. Não aplique regras monetárias de arredondamento não especificadas.

R3 — Reservar estoque antes de cobrar

Chame `reservar(sku, quantidade)` uma única vez. Se retornar `false`, lance `EstoqueInsuficienteException`. Não cobre, não registre o pedido e não envie notificação. Também não chame `liberar`, pois nenhuma reserva foi efetuada.

R4 — Compensar uma cobrança recusada

Se o estoque foi reservado, chame `cobrar(clienteId, total)` uma única vez. Se receber `new Pagamento(false, null)`, libere a reserva uma única vez com o mesmo SKU e a mesma quantidade e lance `PagamentoRecusadoException`. Não registre pedido e não envie notificação.

A ordem relevante é: **reservar → cobrar → liberar**.

R5 — Compensar uma falha técnica de cobrança

Se `cobrar` lançar `FalhaPagamentoException`, libere a reserva uma única vez e propague **a mesma instância da exceção**, sem substituí-la por uma exceção genérica. Não registre pedido e não envie notificação.

Não trate a recusa de negócio de R4 como se fosse a falha técnica de R5. São situações diferentes, embora ambas exijam liberar a reserva neste modelo.

R6 — Registrar e notificar uma compra aprovada

Para um pagamento aprovado, registre um `PedidoConfirmado` com todos os dados recebidos, o total calculado e o identificador de transação retornado pelo pagamento. Use o número devolvido por `salvar` ao enviar a confirmação.

A ordem relevante é: **reservar → cobrar → salvar → enviar confirmação**. Cada chamada deve ocorrer exatamente uma vez. A reserva não deve ser liberada. Retorne `ResultadoCheckout` com o número do pedido, o total e `notificacaoEnviada = true`.

R7 — Não desfazer a compra por uma falha de notificação

Se `enviarConfirmacao` lançar `FalhaNotificacaoException`, retorne os mesmos número e total com `notificacaoEnviada = false`. Não lance essa exceção para o chamador, não libere o estoque, não cobre novamente e não registre novamente.

A notificação é um efeito secundário: sua falha não transforma uma compra já aprovada e registrada em uma compra recusada.

Limites do modelo

Assuma que uma reserva bem-sucedida pode ser liberada sem falhas; que o pagamento aprovado possui `transacaoId` válido; e que o repositório devolve um número válido. Falhas de reserva/liberação, falhas do repositório, respostas nulas do gateway, transações distribuídas, estorno financeiro, repetição automática e idempotência estão fora do escopo obrigatório.

A liberação prevista em R5 é uma simplificação didática do contrato local. Em um sistema real, uma falha de comunicação pode deixar o resultado da cobrança incerto; não use esta regra como desenho de checkout para produção.

5. Suíte de testes esperada

Escreva testes para as seguintes famílias de cenários. Você pode dividir uma família em mais de um método; o que importa é a especificação, não uma quantidade artificial de métodos.

1. **Entrada inválida:** solicitação nula; cada um dos três textos nulo, vazio e em branco; quantidade zero e negativa; preço nulo, zero e negativo. São 15 entradas. Use testes parametrizados para evitar duplicação e verifique ausência de interações.
2. **Quantidade mínima válida:** uma unidade deve ser aceita e manter o preço como total.
3. **Cálculo usual:** três unidades de `19.90` devem produzir `59.70` e esse total deve ser enviado ao pagamento.
4. **Precisão:** três unidades de `0.005` devem produzir `0.015`, sem arredondamento.
5. **Sem estoque:** exceção correta, nenhuma cobrança, persistência ou notificação e nenhuma liberação.
6. **Pagamento recusado:** liberação única após a tentativa de cobrança, exceção correta e bloqueio de persistência e notificação.
7. **Falha técnica no pagamento:** liberação única, propagação da mesma exceção e bloqueio dos passos posteriores.
8. **Persistência correta:** capture o `PedidoConfirmado` e confira todos os seis campos. Não verifique apenas se `salvar` recebeu qualquer objeto.
9. **Compra aprovada:** confira os campos do resultado, os argumentos da notificação, a ordem dos quatro efeitos e a ausência de liberação.

10. **Falha de notificação:** resultado com indicador `false`, sem nova cobrança, nova persistência ou liberação.

O JUnit oferece testes parametrizados e asserções de exceções; `assertThrows` devolve a exceção capturada, permitindo verificações adicionais. 2

Em pelo menos um teste, use explicitamente cada um destes recursos do Mockito: `when(...).thenReturn(...)`, `thenThrow(...)`, `doThrow(...)`, `verify(...)`, `never()`, `times(1)`, `verifyNoInteractions(...)`, `ArgumentCaptor` e `InOrder`. Use `@ExtendWith(MockitoExtension.class)` e mocks isolados por teste. Os recursos devem cumprir um propósito real; não basta incluí-los artificialmente. 1

6. Roteiro de TDD

Primeira aula. Comece com o teste vermelho de entrada nula. Implemente a menor validação que o torne verde e refatore sem mudar o comportamento. Amplie a validação com um teste parametrizado. Depois, escreva um teste de compra aprovada configurando apenas as respostas necessárias dos colaboradores. Faça o teste passar e introduza o cálculo exato e a captura do pedido. Não implemente antecipadamente os tratamentos de erro.

Segunda aula. Especifique falta de estoque, recusa de pagamento e falha técnica, um cenário de cada vez. Em cada ciclo, confirme que o novo teste falha pelo motivo esperado antes de alterar a implementação. Por fim, especifique a falha de notificação e use `InOrder` para a sequência exigida pelo contrato.

Ao refatorar, elimine duplicação útil sem esconder as condições de cada cenário. Por exemplo, criar uma solicitação válida em um método auxiliar é razoável; configurar sempre o fluxo inteiro em `@BeforeEach` não é, pois os testes de erro não usam todas essas respostas.

Registre **três ciclos reais de Red–Green–Refactor**. Para cada ciclo, entregue o nome do teste, a falha observada, a mudança mínima que o tornou verde e a refatoração realizada — ou a justificativa de que nenhuma refatoração era necessária. Podem ser utilizados pequenos diffs, histórico de commits ou registros de execução. Uma suíte pronta e verde, sozinha, não demonstra que foi desenvolvida por TDD. Não reconstrua artificialmente esse histórico depois da implementação.

7. Pistas sobre mocks — sem a solução

Configure apenas as respostas consumidas no cenário. Um método `void` de um mock não precisa de `doNothing()` para o caminho normal; para simular a falha da notificação, use `doThrow(...).when(notificador)...`. 1

Use argumentos específicos quando eles exprimirem uma regra. Se utilizar um matcher como `any(...)` ou `eq(...)` em uma chamada com múltiplos parâmetros, utilize matchers para todos os parâmetros dessa chamada. Um `ArgumentCaptor` pode revelar se o serviço construiu corretamente o pedido entregue ao repositório. 1

`verifyNoInteractions(pagamento, repositorio, notificador)` é útil após uma falta de estoque. Já `verify(estoque, never()).liberar(...)` expressa que essa operação particular não é permitida, sem proibir a reserva. Verificar todas as chamadas privadas ou a ordem de operações que não fazem parte do contrato tornaria a suíte frágil; aqui, a ordem dos efeitos externos é uma exigência explícita. 1

Use objetos reais para `SolicitacaoCheckout`, `Pagamento`, `PedidoConfirmado`, `ResultadoCheckout` e `BigDecimal`. **Não crie um mock de `ServicoCheckout`**, não espione o serviço e não simule seu método `finalizar`: é justamente o comportamento real dele que está sendo testado.

Lembre que um mock não configurado pode devolver `false` ou `null`. Se seu teste cair inesperadamente no caminho de falta de estoque, verifique se a reserva foi configurada antes de enfraquecer a asserção. Não resolva problemas de configuração usando `lenient()` indiscriminadamente ou capturando qualquer `Exception` na implementação.

Para comparar valores de `BigDecimal`, considere que `equals` também observa a escala: `59.70` e `59.7` não são iguais por esse método. Neste exercício, os exemplos preservam a escala da multiplicação. Quando a intenção de uma asserção for apenas igualdade numérica, você também pode verificar que `esperado.compareTo(obtido) == 0`.

8. Entrega e avaliação

Entregue o código-fonte, os testes executáveis com `mvn test`, os registros dos três ciclos de TDD e um texto curto respondendo: o que foi mockado e por quê? Qual teste verifica estado e qual verifica interação? Que defeito seria revelado pelo captor? Por que verificar a ordem aqui não é apenas verificar um detalhe de implementação?

A avaliação vale **10 pontos**:

- **Mocks e interações — 5 pontos:** configuração pertinente e argumentos corretos, 1 ponto; ausência de efeitos proibidos, 1 ponto; captura e conferência do pedido, 1 ponto; ordem e liberação de reserva, 1 ponto; simulação das duas falhas técnicas e verificação de seus efeitos, 1 ponto.
- **Testes de unidade — 2 pontos:** entradas inválidas parametrizadas e limites, 1 ponto; resultados, exceções e precisão numérica, 1 ponto.
- **Processo TDD — 2 pontos:** evidência de três ciclos genuínos, 1 ponto; incrementos pequenos e refatorações justificadas, 1 ponto.

- **Implementação — 1 ponto:** cumprimento dos contratos, injeção por construtor e legibilidade, sem infraestrutura externa nem regras adicionais que alterem o comportamento exigido.

Não é obrigatório produzir uma classe de teste para cada interface. Elas são fronteiras simuladas, não implementações sob teste. Cobertura de linhas pode apoiar a revisão, mas não substitui asserções úteis nem verificações dos efeitos proibidos.

9. Desafio opcional: descobrir um teste fraco

Depois de a suíte passar, altere temporariamente uma regra na implementação: notifique antes de salvar, use outro número na notificação, remova a liberação após recusa ou arredonde `0.015` para duas casas. Identifique qual teste fica vermelho. Se nenhum falhar, explique a lacuna e melhore a suíte antes de desfazer a alteração.

Essas alterações são experimentos locais; não devem permanecer na entrega. Não é necessário instalar uma ferramenta de mutation testing para realizar este desafio.

Referências

[1] Mockito 5.18.0 — configuração, stubbing e verificação
[2] JUnit 5 User Guide — testes parametrizados, exceções e Maven